

Basic Timestamp Ordering Protocol

Database Management Systems — Numerical Solution Guide & Rules

1. Fundamental Concepts & Initial State

Each transaction T_i is issued a unique monotonically increasing timestamp $TS(T_i)$ upon entering the system. Older transactions have smaller timestamp values.

Transaction Timestamps (Example)

- $TS(T_1) = 100$ (Oldest)
 - $TS(T_2) = 200$ (Younger)
 - $TS(T_3) = 300$ (Youngest)
- Target Priority Order: $T_1 \rightarrow T_2 \rightarrow T_3$

Data Item Timestamps

For each data item X , maintain two values:

- $RTS(X)$: Largest TS of any transaction that read X successfully.
- $WTS(X)$: Largest TS of any transaction that wrote X successfully.

Initial Value for all items: $RTS = 0$, $WTS = 0$

2. Protocol Rules (MUST MEMORIZE)

RULE 1: READ Operation — $R(X)$ by Transaction T_i

When transaction T_i attempts to execute $R(X)$:

- If $WTS(X) > TS(T_i) \rightarrow$ **ROLLBACK T_i** (A younger transaction already wrote X)
- Else $\rightarrow$ **EXECUTE $R(X)$** and update: $RTS(X) = \max(RTS(X), TS(T_i))$

RULE 2: WRITE Operation — $W(X)$ by Transaction T_i

When transaction T_i attempts to execute $W(X)$, check **two conditions**:

1. If $RTS(X) > TS(T_i) \rightarrow$ **ROLLBACK T_i** (A younger transaction already read X)
2. Else if $WTS(X) > TS(T_i) \rightarrow$ **ROLLBACK T_i** (A younger transaction already wrote X)
3. Else $\rightarrow$ **EXECUTE $W(X)$** and update: $WTS(X) = TS(T_i)$

Memory Shortcut & Core Logic

- **READ** conflicts with **WRITE** $\rightarrow$ Check **WTS** only.
- **WRITE** conflicts with **READ & WRITE** $\rightarrow$ Check **RTS + WTS**.
- **Key Principle**: If a *younger* transaction has already performed a conflicting operation, an *older* transaction attempting the operation later **MUST** roll back.

3. Comprehensive Numerical Example

Given: $TS(T_1)=100$, $TS(T_2)=200$, $TS(T_3)=300$. Initial $RTS=0$, $WTS=0$ for A, B, C.

Op 1: $T_1 \rightarrow R(A)$

SUCCESS

Check: $WTS(A) > TS(T_1) \rightarrow 0 > 100$ (False). Update: $RTS(A) = \max(0, 100) = 100$.

Op 2: $T_2 \rightarrow R(B)$

SUCCESS

Check: $WTS(B) > TS(T_2) \rightarrow 0 > 200$ (False). Update: $RTS(B) = \max(0, 200) = 200$.

Numerical Example (Continued)

Op 4: T3 → R(B)

SUCCESS

Check: $WTS(B) > TS(T3) \rightarrow 0 > 300$ (False). Update: $RTS(B) = \text{MAX}(200, 300) = 300$.

Op 5: T1 → R(C)

SUCCESS

Check: $WTS(C) > TS(T1) \rightarrow 100 > 100$ (False, strictly greater $>$ required). Update: $RTS(C) = \text{MAX}(0, 100) = 100$.

Op 6: T2 → W(B)

ROLLBACK T2 ❌

Check 1: $RTS(B) > TS(T2) \rightarrow 300 > 200$ (TRUE). T3 (younger) already read B. T2 rolls back, remaining operations of T2 aborted.

Op 7: T3 → W(A)

SUCCESS

Check 1: $RTS(A) > TS(T3)$ ($100 > 300$ False) | Check 2: $WTS(A) > TS(T3)$ ($0 > 300$ False). Update: $WTS(A) = 300$.

4. Final Database Timestamp Table

Data Item	Read Timestamp (RTS)	Write Timestamp (WTS)	Status / Last Modifying Operation
A	100	300	Written by T3 (Op 7)
B	300	0	Read by T3 (Op 4); T2 Write failed
C	100	100	Written by T1 (Op 3), Read by T1 (Op 5)

5. Quick 4-Step Solution Algorithm

Step 1: Identify Target

Note transaction T_i and its fixed timestamp $TS(T_i)$.

Step 2: Check Conditions

- R(X) : Check $WTS(X) > TS(T_i)$
- W(X) : Check $RTS(X) > TS(T_i)$ & $WTS(X) > TS(T_i)$

Step 3: Evaluate

If any condition is **TRUE**, abort & **ROLLBACK T_i** .

Step 4: Update Table

If **FALSE**, execute operation and update **RTS** or **WTS** to new max.

6. Decision Flowchart

